

COSC2804 Assignment 3: Generating and Solving Mazes



[!CAUTION] **DO NOT CHANGE THIS FILE** 🚫 and **READ IT IN FULL & WITH ATTENTION TO DETAIL.** 👁️

- **Course:** COSC2804 C++ Programming Studio
- **Assignment type:** Groups of two students by default
- **Marks:** 50 marks total (see [Deliverables and Marking](#) below)

[!IMPORTANT] 🚫 **About this repo:** You must ALWAYS keep your fork **private** and **never share it** with anybody in or outside the course, *even after the course is completed*. You are not allowed to make another repository copy outside the provided GitHub Classroom without the written permission of the teaching staff. Please respect the author's requests and other students' learning journeys.

Course Learning Outcomes

This assignment covers the following course learning outcomes:

- [CLO1] Analyse and solve computing problems; design and develop suitable algorithmic solutions; implement and debug algorithmic solutions using modern skills and practices in the C++ programming language;
- [CLO3] Demonstrate the ability to communicate effectively with industry professionals and peers;
- [CLO4] Demonstrate skills for self-directed learning, reflection and evaluation of your own and your peers work to improve professional practice;
- [CLO5] Demonstrate adherence to appropriate standards and practice of Professionalism and Ethics.

Overview

Some software has strict performance requirements. A robot navigating a warehouse must plan its path fast enough to avoid obstacles in real time. A game engine must render a new frame before the previous one has finished displaying. In domains like these, the choice of language matters: C++ is widely used because it gives programmers direct control over memory and execution, with no hidden costs from a garbage collector or runtime. This assignment introduces you to that style of programming.

For a real-world example of how much performance matters, take a look at [Micromouse \(Veritasium video\)](#) — a competition where small autonomous robots must solve a physical maze as fast as possible, on minimal hardware.

You will build a programs that generate and solve mazes, and render them in Minecraft. The project is structured in two phases.

Phase 1 treats the generator and solver as independent tools that communicate through text. You run them in sequence on the command line, and the output of one becomes the input of the next. This is a well-established approach to building composable tools, and it will allow you to test your components in isolation.

Phase 2 combines both components into a single program that passes data directly through C++ objects, with no text conversion in between. You will benchmark both approaches and explain the difference in your report.

This is a pair project. One student is responsible for the maze generator; the other for the solver. Both students collaborate on the integration and the Minecraft visualiser.

Deliverable Programs

You will build four C++ programs:

Program	What it does
<code>maze-gen</code>	Generates a random perfect maze using the Recursive Division algorithm and prints it to <code>stdout</code> as an ASCII grid.
<code>maze-solve</code>	Reads a maze from <code>stdin</code> , traces a path from S (start) to E (end), and prints the solved grid to <code>stdout</code> with visit-count markers on every cell the solver stepped on.
<code>maze-view</code>	Reads a maze from <code>stdin</code> and renders it as a 3D maze inside Minecraft. Produces no text output — it communicates directly with the Minecraft server via the mcpp library.
<code>maze-final</code>	Integrates generation and solving in a single binary. Instantiates <code>Maze</code> and <code>Solver</code> objects directly in memory and passes data via C++ interfaces — no ASCII serialisation, no pipes.

You will also produce the following documents:

Document	Owner	Contents
<code>TESTING.md</code>	Individual	Black-box test cases, CLI commands to reproduce them, and an explanation of the termination-bound logic.
<code>REPORT.md</code>	Group	C++ interface documentation (<code>Maze/Solver</code> public API), benchmark results, and the video link .
<code>CONTRIBUTIONS.md</code>	Group	Each student's full name, student ID, GitHub username, and a paragraph summarising their individual contribution.

[!NOTE] This assignment is a pair project. Two students *own* separate modules — Student 1 owns `Maze` and `maze-gen`; Student 2 owns `Solver`, `Path`, and `maze-solve`. These two modules must interoperate via agreed contracts: the ASCII maze format, the C++ class interfaces, and the CLI behaviour. You must be able to defend your own code and write black-box tests *individually*, and also demonstrate that the pieces integrate correctly.

Phase 1: The Unix Pipeline Solution

`maze-gen`, `maze-solve`, and `maze-view` communicate exclusively via `stdin` and `stdout`, and can be chained with a shell pipe:

```
# Generate and validate a maze (should always be VALID for any seed)
./maze-gen 3 3 1 | ./maze-gen --validate
./maze-gen 5 5 42 | ./maze-gen --validate

# Generate, solve, and render a 21x21 maze in Minecraft
./maze-gen 21 21 99 | ./maze-solve --alg right | ./maze-view

# Save a maze to file, then solve it separately
./maze-gen 11 11 99 > maze.txt
./maze-solve --alg right < maze.txt
```

`maze-view` sits at the end of that chain as a *sink*: it reads a maze from `stdin` but its output is the 3D world that appears in Minecraft, not text on `stdout`. Swapping in a better solver, or a different generator, requires changing only one stage of the pipe — neither end needs to know the other changed.

The ASCII format passed between programs looks like this — both columns show `stdout` only:

```
./maze-gen 5 5 1  ./maze-gen 5 5 1 | ./maze-solve --alg right
```

```
XXXXX
S X X
X X X
X   E
XXXXX
```

```
XXXXX
S1X X
X1X X
X111E
XXXXX
```

Each digit on the solved grid is the visit count for that cell — `1` means visited exactly once. The solver also writes `PATH FOUND` and the step trace to `stderr`, which is not shown here.

[!NOTE] All four programs send primary output (the maze grid) to `stdout` and all diagnostic messages to `stderr`. This keeps the pipe clean — `stderr` goes to your terminal and never contaminates the data stream. Common redirects: `> file` (stdout), `2> file` (stderr), `2>&1` (merge), `>/dev/null 2>&1` (discard all).

Phase 2: The Integrated C++ Solution

The text pipeline has a measurable cost: every maze must be formatted as ASCII on one end, transmitted through a pipe, and re-parsed on the other.

`maze-final` removes the pipeline: it instantiates `Maze` and `Solver` objects in the same program and passes data via C++ method calls, with no text serialisation or inter-process communication.

Data Structures and General Requirements

This project requires you to build your own types using primitive C++: raw pointers, arrays, and explicit `new/delete`. STL containers (`<vector>`, `<list>`, etc.) are **forbidden for core logic**. Every allocation must be explicitly freed, with zero memory leaks at program termination.

- **Destructors.** Any class that allocates heap memory must implement a destructor that frees it. You are not required to implement copy constructors or copy-assignment operators unless your design *specifically needs them* — but if you do allocate memory in a class and find yourself copying objects, you will need to handle that correctly.
- **Custom Path.** The solver's path must be recorded in a custom `linked-list` type — a chain of `PathNode*` pointers allocated on the heap — not `std::list` or any other container.
- **Immutable grid (`const Maze&`).** The maze is passed to the solver at construction as a `const` reference and must not be modified. If the solver needs to track state (e.g. visit counts or a working copy), it must allocate that separately. This design constraint is explained in detail in [Phase 2 — In-Memory Integration](#).
- **Recursive generator.** The maze generator must be implemented using recursion. The Recursive Division algorithm is inherently recursive; an iterative approach is not acceptable.
- **Abstract solver hierarchy.** The solver must be designed as an abstract class hierarchy. `Solver` is the pure-virtual base class; `WallFollower` holds the shared wall-following logic; `RightHandSolver` and `LeftHandSolver` extend it with their respective directional bias. You design the internals — the class structure and the `const Maze&` constructor are the fixed constraints.
- **Extension solvers exception.** You are free to implement additional algorithms beyond `RightHandSolver` and `LeftHandSolver` (e.g. A*, BFS) as separate classes that inherit from `Solver`. These will not be graded but may be toggled in `maze-final` via the `--alg` flag.

Program Specifications

Your repository must include a `Makefile` that compiles all four programs. Markers will build your project by running `make` from the repository root — no other build steps should be required. The starter `Makefile` is provided and should not need to be restructured, but you must ensure it continues to build all four targets correctly as you add source files. You are welcome to add additional targets for testing or benchmarking.

Compiler flags used to build and validate your submission:

```
CXX      = g++
CXXFLAGS = -Wall -Werror -std=c++17 -g -O
```

`-Wall -Werror` means **all warnings are treated as errors** — code that produces compiler warnings will not build. `-std=c++17` is the required language standard. `-g` includes debug symbols (required for `valgrind/leaks` runs). `-O` enables basic optimisation.

Phase 1: The Unix Pipeline

1. `maze-gen` — Maze Generator

- **Usage (generation):** `./maze-gen <width> <height> <seed>`
- **Usage (validation):** `./maze-gen --validate`

Generation mode produces a random perfect maze using the **Recursive Division** algorithm (described in [Walter Pullen's algorithm references](#); full walkthrough in [docs/recursive-division.md](#)) and prints the result as an ASCII grid to `stdout`. A perfect maze has exactly one path between any two cells and contains no loops. **Your implementation must use recursion** — the algorithm is inherently recursive and an iterative approach is not acceptable.

Placement rules for the entrance and exit:

- The Start (**S**) is placed on the **left outer wall**, at the first cell adjacent to a reachable interior passage, scanning top-to-bottom.
- The Exit (**E**) is placed on the **right outer wall**, at the first cell adjacent to a reachable interior passage, scanning bottom-to-top.
- Start and Exit are always on **opposing sides** of the maze, guaranteeing the solver must traverse the full interior.

All errors, warnings, and debug output go exclusively to `stderr`. `stdout` carries only the maze grid.

Defensive constraints: To assist with the correct generation of mazes, `width` and `height` must be **odd numbers ≥ 3** . Any other input must print an error to `stderr` and exit cleanly.

Validation mode (`--validate`) reads an ASCII maze from `stdin` and runs a **Flood Fill** to verify that all open cells are connected and that there are no loops. It prints exactly **VALID** or **INVALID** to `stderr`; nothing is written to `stdout`. See [docs/imperfect-mazes.md](#) for a description of both failure modes and hints for implementing the check. Consider how this utility can be used in black-box tests to verify that `maze-gen` is producing valid mazes for any seed.

2. `maze-solve` — Maze Solver

- **Usage:** `./maze-solve [--alg right | left | <custom>]`

[!TIP] Reading the maze from `stdin` into a 2D structure is the first thing you need to do. `std::vector<std::string>` and `std::getline` make this straightforward — `main` is glue code, not a core data structure, so STL is fine here:

```
#include <string>
#include <vector>

std::vector<std::string> lines;
std::string line;
while (std::getline(std::cin, line)) {
    lines.push_back(line);
}
int h = lines.size();
int w = lines[0].size();
```

EOF (End of file) is signalled automatically when the pipe closes, or by **Ctrl+D** when typing interactively.

Reads an ASCII maze from `stdin` and traces a path from **S** to **E** using the chosen algorithm. You are required to implement **both** wall-follower directions:

- `--alg right` — **Right-Hand Rule** (default): always try to turn right first.
- `--alg left` — **Left-Hand Rule**: always try to turn left first.

Both must be implemented as subclasses of a shared `WallFollower` intermediate class — **the algorithm must not be duplicated** between `RightHandSolver` and `LeftHandSolver`. Additional custom algorithms (e.g. A*, BFS) may be added as further subclasses and selected via `--alg <custom>`, but are not assessed.

See [Walter Pullen's algorithm references](#) for a detailed description of the wall-follower algorithm.

The solved maze is printed to `stdout`, with each cell on the solution path replaced by a digit indicating how many times the solver stepped on it (**1, 2, 3...**). The status line is printed to `stderr` first, followed by the step-by-step coordinate history:

```
PATH FOUND
(0,1) -> (1,1) -> (2,1) -> ...
```

`PATH FOUND` or `NO PATH FOUND` must be the **first** line written to `stderr`. The remainder of `stderr` is the coordinate trace. A worked example of a solved maze is given in [docs/solved-example.md](#).

Safety & termination: The solver must never infinitely loop. An upper bound iteration limit (e.g. `width × height × 4` steps) must detect an unsolvable maze or a trapped start, print `NO PATH FOUND` to `stderr`, and exit with code **1**. You are welcome to determine failure sooner if your algorithm allows it (e.g. detecting a trapped start with no adjacent passages), but the iteration bound is a required safety net.

Exit codes. Programs must exit with a non-zero code whenever they cannot complete their primary task:

Program	Condition	Exit code
<code>maze-gen</code>	Invalid dimensions (not odd, or <code>< 3</code>)	1
<code>maze-solve</code>	Empty or missing <code>stdin</code>	1
<code>maze-solve</code>	Iteration bound exceeded (no path)	1
<code>maze-final</code>	Wrong number of arguments	1
<code>maze-final</code>	Invalid dimensions	1
<code>maze-final</code>	Iteration bound exceeded (no path)	1

All other exits (successful solve, successful generation) must return **0**.

3. `maze-view` — Minecraft Visualiser (*shared task*)

- **Usage:** `./maze-view`

Reads an ASCII maze (solved or unsolved) from `stdin` and renders it as a 3D structure inside Minecraft, starting at the player's current position. After construction, the **player is teleported to the maze entrance** (`S`). `maze-view` produces **no text output** — all communication is handled directly via the `mcpp` library.

Block choice is up to you, provided the rendered result clearly conveys the **maze structure** (walls vs passages) and the **visit-frequency counts** on solved cells (the digit overlay from `maze-solve`). The table below shows one possible mapping:

ASCII char	Suggested Minecraft block	Meaning
X	Oak log + oak leaves (3 blocks tall)	Wall
S	Gold block (under floor)	Start / entrance
E	Diamond block (under floor)	Exit
1	Lime carpet	Visited once
2	Yellow carpet	Visited twice
3–9	Red carpet	Visited 3+ times

[!NOTE] **Leaf decay:** In vanilla Minecraft, oak leaves placed without an adjacent log will decay shortly after the chunk loads. If you want a hedge aesthetic, place an oak log inside each wall column beneath or within the leaf stack to prevent decay. Any non-decaying block (e.g. green concrete, mossy cobblestone) avoids this.

Phase 2: Native C++ Integration

4. `maze-final` — In-Memory Integration

- **Usage:** `./maze-final <width> <height> <seed> [--alg right | left]`

Combines generation and solving in a single binary. A `Maze` object is generated and then passed directly to a `Solver` by **`const` reference** — no text serialisation, no pipes, no re-parsing.

`maze-final` produces identical output to the pipeline — the same `stdout` and `stderr` as `maze-solve` — so you can run both side-by-side and compare results directly:

- **`stdout`:** the solved ASCII grid, with visit-count digits on the solution path (identical format to `maze-solve`).
- **`stderr`:** `PATH FOUND` or `NO PATH FOUND` as the first line, followed by the coordinate trace (identical format to `maze-solve`).

This makes timing comparisons apples-to-apples: the only difference is how the maze data moves from generator to solver.

The `const Maze&` interface contract. The maze is passed to the solver at construction as a `const` reference and must not be modified. Any working state (visit counts, a mutable grid copy) must be

allocated separately inside the solver. Your `Solver` class must enforce this contract:

```
class Solver {
public:
    Solver(const Maze& maze); // const: read-only – allocate any working
state separately
    virtual bool solve() = 0; // pure virtual: implemented by each concrete
solver
    virtual ~Solver() {}
};
```

You design the rest. Add whatever member variables and helper methods your implementation needs. Any concrete solver (e.g. `RightHandSolver`) inherits from `Solver` and receives the maze at construction. If it needs a mutable working copy, it constructs one explicitly — either by invoking the `Maze` copy constructor or by allocating a separate `int*` array. The solver must never modify the original maze data.

Task Allocation

This is a pair project. To minimise merge conflicts, primary responsibilities are separated by component:

- **Student 1:** Implements `maze-gen`. Responsible for the `Maze` class (primitive dynamic arrays, destructor), the recursive division generation algorithm, and the `--validate` flood-fill mode (see [docs/imperfect-mazes.md](#)). Also responsible for writing the black-box test cases for `maze-gen` and `--validate`, documented in `TESTING.md`.
- **Student 2:** Implements `maze-solve`. Responsible for the abstract `Solver` class hierarchy, the custom `Path` linked-list data structure, the `WallFollower` intermediate class (Template Method pattern), both the `RightHandSolver` and `LeftHandSolver` implementations, solver termination safety, and visit-frequency counting logic. Also responsible for writing the black-box test cases for `maze-solve`, documented in `TESTING.md`.
- **Shared:** Both students collaboratively implement `maze-final` and `maze-view`. `maze-view` is a joint creative and technical exercise — you are encouraged to work together on the rendering design. Both students must be able to explain and defend all shared code individually.

[!WARNING] If a student deviates from their assigned tasks and their individual contribution is found to be below the expected standard, their grade may be significantly reduced, regardless of the overall quality of the submission.

Other Requirements

Your repository must contain the following, and the video must be linked from `REPORT.md`.

TESTING.md

A testing methodology document that serves as a marker-facing record of every test you have written. It must contain:

- A description of every black-box test case: the edge condition it targets, what the expected `stdout` output is, and what the expected `stderr` output is.
- Exact CLI commands to reproduce every test (these commands must also be shown during the demonstration video).
- An explanation of how the solver's infinite-loop termination works, and which test cases exercise that bound.

What to test. Tests must verify both `stdout` and `stderr`. Required cases include, at minimum: valid maze generation, invalid dimensions, the solver reaching the exit, the solver failing on an unsolvable layout, a trapped start, and `--validate` on both valid and invalid mazes.

How to test. Use `diff` to compare program output against expected files. You can capture `stdout` and `stderr` separately:

```
# Capture stdout and stderr into separate files
./maze-gen 7 7 42 > out.txt 2> err.txt

# Compare against expected output
diff out.txt tests/test_gen_basic.expected_out
diff err.txt tests/test_gen_basic.expected_err
```

Pipes can also be used directly in tests — for example, to verify that a generated maze is always valid:

```
# This should print VALID to stderr and exit 0
./maze-gen 11 11 99 | ./maze-gen --validate
```

Any additional test methodology or tooling you develop should be documented in `TESTING.md`.

You are welcome to add test targets to your `Makefile` (e.g. `make test`) to speed up your test runs. This is good practice and can make your workflow much faster, but it is not required.

REPORT.md

Document the C++ interface used by `maze-final`. Detail how data flows between `Maze` and `Solver`, and list the public methods exposed for traversal (e.g. `isWall()`, `getStart()`, `getWidth()`).

Include the **video link** in `REPORT.md` — add a section at the top or bottom with the URL so markers can find it immediately.

Benchmarks. Measure and report that native in-memory integration outperforms the text pipeline. The goal is to show a clear trend as maze size grows — the choice of sizes and number of iterations is up to you, and will depend on how fast your machine is and how your implementation scales. Pick values of `$N$` where the difference is visible. The commands below show the pattern; substitute your chosen `$N$`:

```
# Unix Pipeline Benchmark
time bash -c 'for i in $(seq 1 100); do ./maze-gen N N 123 | ./maze-solve
2>/dev/null; done'
```

```
# Integrated C++ Benchmark
time bash -c 'for i in $(seq 1 100); do ./maze-final N N 123 2>/dev/null;
done'
```

`2>/dev/null` suppresses `stderr` (the path trace) so it does not dominate timing. Include your timing outputs and a technical explanation of *why* the integrated version is faster in `REPORT.md`.

Video Demonstration

Submit a single video covering both students' work. Each student should speak for around **5–8 minutes** (total: 10–20 minutes). Both students must present.

Required content for each student:

- **Top five design decisions.** The most important design choices made to ensure correctness, efficiency, or testability — e.g. data structure decisions, how `const Maze&` enforces the solver contract, how infinite loops are detected and terminated, how the test suite achieves coverage, or any optimisations made.
- **Test walkthrough.** A general overview of the black-box tests written, with at least one live demo per student. Include guidance on how to run any more complex test tooling.
- **Memory check.** Run `valgrind --tool=memcheck` (Linux) or `leaks` (macOS) live on screen and confirm zero leaks.
- **Minecraft demonstration.** At least one student must show `maze-view` in action with a live screen capture of the rendered maze, entrance, exit, and solved path frequency colours.
- **Benchmark results.** Walk through the numbers from `REPORT.md` and explain the trend as `$N$` grows.

Recording format: Use screen capture for terminal sessions and the Minecraft world. Microsoft Teams, OBS Studio, or QuickTime are all acceptable.

Submitting: Include the video link in `REPORT.md`. Ensure the video is accessible to anyone with the link (e.g. unlisted YouTube, Google Drive with link sharing) and requires no special software or accounts.

Checkpoints

A checkpoint takes place in the third class of each project week. Bring your current implementation and be ready to demo it to your tutor and explain the contribution of each team member — including showing your commit history if asked.

The milestones for each week are:

- **Week 5:** Understand specification; agree on ASCII format; initial black-box tests drafted and documented in `TESTING.md`.
- **Week 6:** Standalone `maze-gen` (including `--validate`) and `maze-solve` implemented and piping correctly. Agree on interface for `Maze` objects.
- **Week 7:** `maze-final` and `maze-view` implemented; zero memory leaks confirmed.

- **Week 8:** Benchmarks runs and optimisations; **REPORT.md** finalised; demonstration videos recorded.

Checkpoints do not carry marks directly, but the teaching team record observations at each session for your group. Those observations — whether you were on track, the depth of your answers, and the evidence of individual contribution — are used by markers to inform their judgment on your final submission. Attending all checkpoints is strongly advised. *Not attending any sessions to answer questions about your implementation, or not being able to explain your contribution, may lead to a significant penalty.*

GitHub Contributions

Markers will review your individual commit history. The following rules apply without exception:

- **Zero commits = zero marks for the entire assignment.** There is no appeal for this.
- Thin or uneven contribution will be penalised proportionally. Do not email code to your partner and ask them to commit it on your behalf — this is not a valid excuse.
- Commit early and commit often. Your history should show incremental progress across the project, not a small number of large commits near the deadline.
- Write meaningful commit messages. Messages like **fix**, **work**, or **changes** are unacceptable. Each message should clearly summarise what was changed and why — for example: **Implement flood-fill isolation check in --validate mode.**
- Pair programming is encouraged, but all team members must take turns as the driver and commit. Where both students contributed to a change, include both names in the commit message: e.g., **Fixed outer wall removal bug. Contributors: Anna and Tom.**
- If your GitHub username does not match your name or student ID, document this in **CONTRIBUTIONS.md**.
- If work is spread across multiple branches, document this in **CONTRIBUTIONS.md** so markers do not miss contributions.

CONTRIBUTIONS.md must be present in the root of your repository and contain:

- Each student's full name, student ID, and GitHub username.
 - A brief summary of who implemented what (one paragraph per student is sufficient).
-

What to Submit

Your submission is your GitHub Classroom repository. Everything markers need must be on the **main branch** before the deadline. Ensure the following are present:

File / Folder	Notes
Makefile	Must build all four binaries with make from the repo root.
Source files (*.h , *.cpp)	All files needed to compile the four programs.
tests/	All test input and expected-output files.

File / Folder	Notes
TESTING.md	Full test documentation — see Testing .
REPORT.md	Interface documentation, benchmark analysis, and video link .
CONTRIBUTIONS.md	Lists each student's real name, student ID, GitHub username, and a brief summary of their individual contribution. See GitHub Contributions .

While the Makefile must work from the repo root, you are free to organise your source files in subdirectories if you wish — just ensure the Makefile is updated accordingly. Likewise, you can change where your targets are built. Always ensure that the clean target works correctly to remove all binaries and object files.

[!IMPORTANT] Submissions missing [REPORT.md](#) or [CONTRIBUTIONS.md](#) **will lose marks** in the corresponding rubric categories regardless of the quality of the code.

Deliverables and Marking (50 Marks)

Individual: 15 + Testing: 8 + Integration: 12 + Report: 5 + Video: 10 = **50 marks**

Individual Implementation (15 marks) — *assessed individually*

Each student is responsible for implementing the functionality assigned to them (see [Task Allocation](#)). Assessment focuses on correctness of input/output behaviour, appropriate data structures, adherence to the memory and design constraints, and zero memory leaks.

Marks	Descriptor
0	No attempt, or the program does not compile.
4	Code compiles; partial implementation exists but key input/output behaviours are missing or incorrect.
9	Core functionality is implemented and produces broadly correct output, but there are logical errors, missing edge cases, or constraint violations (e.g. STL containers used for core logic, destructor not implemented).
13	Functionality is complete and correct for standard cases but has gaps in edge-case handling, efficiency, or memory management (leaks detected by valgrind/leaks).
15	Fully correct input/output behaviour across all cases. Appropriate data structures. All memory constraints met. Zero leaks confirmed.

Testing (8 marks) — *assessed individually*

Each student must create black-box test cases covering the components they are responsible for. Tests must be placed in [tests/](#), with descriptive filenames, and fully documented in [TESTING.md](#). Tests must verify both [stdout](#) and [stderr](#) output.

Marks	Descriptor
0	No tests, or tests do not follow the <code>.in/.expected</code> format or cannot be reproduced from <code>TESTING.md</code> .
3	Tests cover standard cases but miss important edge cases (e.g. invalid dimensions, trapped starts, unsolvable mazes).
6	Most edge cases are covered; <code>TESTING.md</code> explains the purpose of each test.
8	Comprehensive coverage of normal and edge cases for all assigned components. Both <code>stdout</code> and <code>stderr</code> are verified. <code>TESTING.md</code> is complete and commands are reproducible.

Integration (12 marks) — *assessed as a group*

Both students must integrate all components into `maze-final` and complete `maze-view`. `maze-final` must construct the `Solver` with a `const Maze&` — the agreed interface contract — with no text serialisation between the two. Zero memory leaks must be confirmed by `valgrind --tool=memcheck` (or `leaks` on macOS). `maze-view` must correctly render the maze structure and visit-frequency counts in Minecraft and teleport the player to the entrance. Both `--alg right` (`RightHandSolver`) and `--alg left` (`LeftHandSolver`) must work correctly in `maze-final`.

[!IMPORTANT] Higher marks are awarded for designs that avoid duplicating the wall-following algorithm. Both solvers **must** share a single implementation via the `WallFollower` intermediate class — copy-pasting the algorithm into each subclass will be penalised as poor design.

Marks	Descriptor
0	Programs do not compile, or key components are absent.
4	Programs compile and basic integration works, but <code>maze-final</code> does not honour the <code>const Maze&</code> contract (e.g. serialises through text), or <code>maze-view</code> does not render correctly.
7	Integration is functionally correct but: memory leaks are present, or only one solver direction works, or <code>RightHandSolver/LeftHandSolver</code> duplicate algorithm code rather than sharing it via <code>WallFollower</code> .
10	Correct integration with zero memory leaks and a clear Minecraft render; both <code>--alg right</code> and <code>--alg left</code> work; solver code is properly shared via <code>WallFollower</code> , but modularisation or efficiency could be improved.
12	Complete, correct integration. <code>const Maze&</code> respected. Zero leaks confirmed. Both solvers implemented without code duplication. <code>maze-view</code> clearly conveys maze structure and solver visit frequencies. Player teleported to entrance.

Performance Report — `REPORT.md` (5 marks) — *assessed as a group*

`REPORT.md` must document the C++ interface used by `maze-final` (public methods, data flow between `Maze` and `Solver`) and present a benchmark analysis comparing the Unix pipeline against native in-memory integration across $N \in \{10, 50, 100, 200\}$. Additionally, if you have implemented any optimisations to improve the performance of `maze-final`, describe those changes and their impact on the benchmark results. We also welcome any suggestions for further optimisations or improvements.

Marks	Descriptor
0	No report, or the report does not address the benchmark or interface design.
2	Benchmark results are included but the explanation is superficial, or the interface is undocumented.
4	Both sections present and mostly correct, but the analysis lacks depth or quantitative support.
5	Clear interface documentation and a thorough evidence-backed explanation of why in-memory integration outperforms the text pipeline as N increases.

Video Demonstration (10 marks) — *assessed individually, submitted and edited together*

Marks	Descriptor
0	Student did not appear, or did not demonstrate their assigned tasks.
4	Work was demonstrated but test walkthroughs were missing, unclear, or the memory check was not shown.
7	All assigned components demonstrated and memory check shown, but the explanation of edge-case coverage or design decisions lacked depth.
10	Clear and complete demonstration of all assigned components and test cases, with proof of zero memory leaks, a compelling Minecraft demo, and a coherent explanation of design decisions and benchmark results.

[!IMPORTANT] **Final grades will reflect individual contributions**, which will be evaluated based on in-class checkpoints and GitHub commit history.

This rubric rewards **demonstrated progress and understanding** — not just starting the task. By this stage of the semester you are expected to have strong C++ programming skills. Minimal or incomplete work will not earn partial credit.